

SMART INDIA HACKATHON 2026

Problem Statement ID: SIH26156 | Category: Software | Theme: Miscellaneous

ULPF

Universal Log Pre-processing Framework

"Different Logs. One Framework. A Safer Tomorrow."

PRODUCT REQUIREMENTS DOCUMENT

PRODUCT REQUIREMENTS DOCUMENT

Product vision, users, scope, features and success metrics — revised to the Local Rule Authoring Agent architecture

Team & Role Allocation

Member	Role	Primary Ownership in This Document
Divyansh Rajat	Team Lead / System Architect	Overall architecture, scope, requirement baseline, traceability
Ashish Bhardwaj	Log Processing Engineer	Ingestion, fingerprinting, rule engine, parser factory
Sakshi	Normalization & Schema Engineer	Unified event model, OCSF/ECS mapping, rule registry, traceability
Parampreet Kaur	Backend & Integration Engineer	APIs, persistence, deployment, downstream connectors
Aryan Jeet	Frontend Engineer	Vendor portal (Studio, Rule Registry, Jobs & Sessions), dashboards
Saksham	Testing, Documentation & Demo	Test strategy, benchmark methodology, acceptance evidence

Reference Documents

Ref.	Document	Source
R1	SIH 2026 Problem Statement SIH26156 — Universal Log Preprocessing Framework	Smart India Hackathon 2026 portal
R2	ULPF Full Understanding & Reference Document (Team S.W.O.R.D.)	Internal design reference — governing document for this baseline
R3	OCSF — Open Cybersecurity Schema Framework	ocsf.io schema specification
R4	ECS — Elastic Common Schema field reference	Elastic documentation
R5	Qwen2.5-Coder technical report	Qwen team / Alibaba, open weights
R6	llama.cpp — local GGUF inference runtime	ggerganov/llama.cpp, open source
R7	RFC 5424 — The Syslog Protocol	IETF
R8	ArcSight Common Event Format (CEF) and IBM LEEF specifications	Vendor specifications
R9	OWASP — Regular expression Denial of Service (ReDoS)	OWASP documentation

1. Executive Summary

Every device in a modern enterprise writes down what it does, and almost none of them write it down the same way. A firewall emits Syslog, an application emits JSON, a legacy appliance emits a pipe-delimited string its vendor invented, and a cloud service emits a nested structure with entirely different field names. All of them may be describing the same login, the same blocked connection, the same policy violation. Before any SIEM, data lake or machine-learning pipeline can reason across them, somebody has to teach the platform what each source means.

That teaching step is manual, source-specific and unbounded. This is the bottleneck ULPF addresses — and it is also the step ULPF now automates through a local, air-gapped Rule Authoring Agent rather than through statistical structure discovery.

ULPF — the Universal Log Pre-processing Framework — is a vendor-neutral, air-gapped preprocessing layer for perimeter security telemetry (firewalls, routers, VPN gateways, IDS/IPS, WAFs, proxies and cloud edge). It stores the exact original event before any transformation, matches it against a versioned Rule Registry by format fingerprint, and — only on a registry miss — invokes a local, quantized LLM to draft a declarative parsing rule. Deterministic tests and mandatory human review decide whether that rule version becomes Active. Approved rules deterministically extract and normalize fields into a canonical ULPF event model, exported as OCSF (primary) or ECS (adapter) to SIEM, data-lake and analytics systems.

The product thesis in one sentence: ULPF does not try to reinvent log parsing or replace a SIEM; it makes heterogeneous log preprocessing lossless, traceable, rule-governed and vendor-independent, with a local AI assistant that turns onboarding a new source into a reviewed, versioned rule-authoring exercise rather than a bespoke development project.

1.1 Product Snapshot

Attribute	Value
Product name	ULPF — Universal Log Pre-processing Framework
Problem statement	SIH26156 (Smart India Hackathon 2026, Software, Miscellaneous)
Product category	Data preprocessing / security data engineering middleware
Position in stack	Between perimeter log sources and SIEM / data lake / AI-ML pipelines — not a SIEM itself
Primary users	SOC analysts, detection/rule engineers, platform and data engineers, compliance officers, vendor integrators
Deployment model	Self-hosted, containerised (Docker Compose); air-gapped by design; local LLM only, no cloud dependency
Core differentiators	Local LLM-assisted rule authoring · Lossless raw evidence · Rule-hash traceability · Versioned rule lifecycle · Air-gapped by design
Target schemas	OCSF as the primary security taxonomy, with an ECS-compatible adapter and vendor extension fields
MVP horizon	Working prototype demonstrating unknown-format onboarding via the Rule Authoring Agent end to end, with measured benchmarks

2. Problem Definition

2.1 Background

Enterprise and government perimeter networks generate log data from firewalls, routers, VPN concentrators, IDS/IPS, WAFs, proxies and cloud edge devices. This data is the raw material of security monitoring, incident

investigation, compliance evidence and operational analytics. The difficulty is not volume alone; it is heterogeneity — the same semantic fact (a source IP, an allow/deny decision) is expressed through different field names, structures and vocabularies across vendors.

2.2 The Operational Bottleneck

The conventional response is a source-specific parser or content pack per source. That approach degrades badly as diversity grows:

- Linear growth of engineering effort — each new source requires a developer to study the format and hand-write extraction logic.
- Fragile to change — a vendor firmware upgrade that renames a field silently breaks downstream detection rules.
- Silent information loss — fields that do not fit the target schema are often dropped during normalization.
- No provenance — once an analyst sees a normalized value, there is often no reliable path back to the exact original record and the exact logic that produced it.
- Vendor lock-in — parsing effort invested in one platform's proprietary configuration does not transfer to another.

2.3 Problem Statement (as addressed)

Design and build a preprocessing framework capable of ingesting logs from perimeter security devices in arbitrary and previously unseen formats, discovering their structure with minimal source-specific development, normalizing them into a common extensible event model, preserving the complete original event and the exact rule provenance of every derived value, and delivering the result to downstream security and analytics platforms — including inside air-gapped environments.

2.4 Why Now

- Open, vendor-neutral event schemas (OCSF, ECS) have matured to the point where a common target taxonomy is realistic rather than aspirational.
- Compact, code-oriented open-weight LLMs (Qwen2.5-Coder-7B and comparable models) now run entirely on-premise via quantized GGUF builds and llama.cpp, fast enough to draft a declarative parsing rule interactively — without sending a single log sample to an external API.
- Because the model only ever emits a schema-validated declarative rule rather than executable code, the classic "can we trust AI-generated logic" objection becomes a structural-validation problem rather than a sandboxing problem.
- Indian public-sector and critical-infrastructure deployments increasingly mandate on-premise or air-gapped operation, which excludes SaaS-first log pipelines and cloud-hosted LLM assistance alike.

3. Product Vision, Positioning & Value

3.1 Vision Statement

A security team should be able to point ULPF at any perimeter log source — including one nobody has seen before — and get back consistent, standardised, analytics-ready events within minutes, with a declarative rule that a human has reviewed and approved, without losing a single byte of the original evidence, and without their data or their sample logs ever leaving their network.

3.2 Positioning

ULPF is deliberately not positioned as a SIEM, and not positioned as a replacement for Splunk, Elastic or QRadar. It is positioned as the vendor-agnostic preprocessing layer that sits in front of whichever of them an organisation already runs.

ULPF is...	ULPF is not...
A preprocessing and normalization framework	A SIEM or a detection engine
A producer of standardised, traceable, rule-governed events	A correlation, alerting or case-management product
A layer that feeds SIEMs, data lakes and ML pipelines	A storage or long-term retention platform
Assisted by a local LLM for drafting new parsing rules	An "AI understands any log automatically" black box
Vendor-independent, self-hosted and air-gapped by default	A managed cloud service

3.3 Value Proposition by Stakeholder

Stakeholder	Pain today	Value delivered by ULPF
SOC analyst	Inconsistent fields across sources make cross-source investigation slow	One consistent OCSF/ECS event shape; one click back to the original raw record and exact rule used
Rule / detection engineer	Every new source needs a hand-written parser before any rule can be written	The local agent drafts a candidate rule; the engineer reviews and approves rather than writing extraction logic from scratch
Platform / data engineer	Parser sprawl and format churn dominate maintenance effort	Rules become versioned, lifecycle-governed configuration instead of bespoke code
Compliance officer	Cannot show that a normalized value faithfully reflects the original record	Every event carries its raw hash, rule ID, rule version and rule hash
Data scientist / ML engineer	Schema inconsistency dominates feature-engineering effort	Typed, normalized OCSF/ECS fields with stable semantics
Air-gapped operator	Most modern pipelines assume cloud or Internet access, including AI-assisted ones	Fully offline operation; the authoring LLM itself runs locally and never calls out

3.4 The Five Product Principles

#	Principle	Engineering meaning
P1	Governed onboarding	An unseen format is onboarded through fingerprint match-or-draft, deterministic validation, and mandatory human review — never through an unreviewed automatic decision
P2	Lossless normalization	Raw bytes are hashed and stored before any parsing; unmapped fields survive as <code>unmapped_fields</code> rather than being dropped
P3	Rule-hash traceability	Every normalized event carries its raw hash, exact rule ID, rule version and rule hash — the provenance chain is a rule reference, not an inferred derivation
P4	Declarative rule safety	The authoring agent emits schema-validated declarative rule JSON, never executable code, shell commands, SQL or network actions — validation is structural, not sandboxing
P5	Air-gapped by design	No mandatory Internet, cloud API or external model call anywhere on the processing or authoring path

4. Target Users, Personas & Journeys

4.1 User Classes

User class	Technical level	Frequency of use	Primary interaction
Security analyst (Tier 1/2)	Medium	Daily	Consumes normalized events downstream; occasionally traces an event back to raw
Rule / detection engineer	High	Weekly	Onboards sources via Studio, reviews LLM-drafted rule candidates, approves or edits them
Platform administrator	High	Weekly	Manages ingestion endpoints, API keys, rule lifecycle, and monitors jobs/sessions
Compliance / audit officer	Low	Monthly / on demand	Retrieves raw evidence and rule-version provenance for audit
Data / ML engineer	High	Continuous	Consumes normalized OCSF/ECS output via API, export or stream

4.2 Personas

Persona A — Meera, Rule / Detection Engineer

Meera maintains detection content for a bank running two SIEMs after an acquisition. Roughly twice a month a new appliance appears on the network. Today the honest answer to "how quickly can you get it in" is one to two days of hand-written parsing. With ULPF, she uploads 3–5 representative samples in Studio, the local agent drafts a candidate declarative rule with parser type, field mappings and required fields, deterministic tests run automatically, and she reviews the diff before approving — typically before lunch.

Persona B — Colonel Rao, Air-Gapped Network Administrator

Rao runs monitoring for an isolated network with no Internet route by policy. Any product that phones home, downloads model weights at runtime, or calls a cloud LLM is disqualified before evaluation. He needs a container bundle he can carry in on physical media, including the GGUF model weights baked into the image, and start with one command.

Persona C — Anil, Compliance Officer

Anil has to demonstrate to an auditor that a normalized event faithfully represents what the device recorded. He does not care how the rule was authored. He cares that for any normalized event, the system can show him the raw bytes, their SHA-256 hash, the exact rule ID and version applied, and that version's approval record.

4.3 Primary User Journey — Onboarding a New Source

Step	Today (without ULPF)	With ULPF
1	Receive sample logs from a new appliance	Receive sample logs from a new appliance
2	Manually inspect and document the format	Upload 3–5 samples in Studio; ULPF fingerprints the format and searches the Rule Registry
3	Write source-specific parser / regex	On a registry miss, the local Rule Authoring Agent drafts a declarative rule (parser type, pattern, field mapping, required fields, masking, fingerprints)
4	Map fields to the platform's schema by hand	The platform validates JSON structure, rejects forbidden content, compiles the parser in a sandbox and runs all test samples
5	Test, iterate, debug edge cases	On validation failure, structured errors are returned to the LLM for at most 2–3 retries

Step	Today (without ULPF)	With ULPF
6	Deploy through change control	A reviewer approves the Draft; the rule version is Activated and becomes reusable immediately
7	Maintain forever; re-fix on format change	Background spot-checks during locked batch/stream ingestion catch drift; a new reviewed rule version is created rather than silently reinterpreting
Time	Typically 1–2 working days	Target: under 10 minutes for the prototype benchmark scenario

4.4 Secondary Journey — Forensic Trace-Back

An analyst investigating an incident sees a normalized event indicating a denied connection. She requests the raw record. ULPF returns the event_id, the original raw bytes from the Immutable Raw Evidence Layer with a verified SHA-256 digest, the exact rule_id, rule_version and rule_hash used, and the schema_version applied. Nothing in that chain required the analyst to know anything about the vendor's format.

5. Goals & Success Metrics

5.1 Product Goals

ID	Goal	Rationale
G1	Reduce source-specific parser development to zero hand-written code for new sources	This is the core bottleneck named in the problem statement
G2	Guarantee complete recoverability of the original event	Forensic and compliance value depends on it
G3	Bind every normalized event to the exact rule ID, version and hash used to produce it	Makes normalization auditable and reproducible rather than opaque
G4	Produce output aligned with an established open schema (OCSF, with an ECS adapter)	Downstream consumption should not require learning a proprietary ULPF schema
G5	Never let an LLM-drafted rule reach production without deterministic validation and human approval	A wrong automatic decision is worse than an unmapped field
G6	Operate fully offline, including the rule-authoring model, in a containerised deployment	Required for the defence, government and regulated deployments this PS targets
G7	Detect and surface format drift in locked batch/stream ingestion without failing the whole job	Silent re-interpretation of a drifted source is worse than a flagged event

5.2 Success Metrics

These are the metrics the prototype is validated against. Values in the target column are engineering targets and acceptance thresholds, not measured results; measured results replace them once benchmarking on the defined hardware and corpus is complete.

ID	Metric	Definition	Target
M1	New-source onboarding time	Elapsed time from first unseen sample to an Activated, reviewer-approved rule	< 10 minutes
M2	Source-specific hand-written code	Lines of bespoke parser code required to onboard a new source	0

ID	Metric	Definition	Target
M3	Rule candidate validation pass rate	LLM-drafted candidates that pass deterministic structural/test validation ÷ candidates drafted	≥ 90%
M4	Raw-event recoverability	Original events byte-identically recoverable with digest verification ÷ events ingested	100%
M5	Required-field completeness	Normalized events with all target-schema-mandatory fields populated ÷ events processed by an Active rule	≥ 95%
M6	Processing throughput	Events successfully normalized per second on defined hardware	Benchmarked
M7	Spot-check drift detection	Probability of detecting at least one drifted event within a locked batch/session, per the adaptive spot-check schedule	Benchmarked

5.3 Anti-Metrics

Numbers the team explicitly refuses to claim, because they cannot be defended:

- "Supports every log format that exists." Universality is measured as coverage across a defined test matrix of formats, structures, transports and schema conditions — not as an unbounded claim.
- "Processes billions of events per day." The prototype measures real throughput on defined hardware; the architecture describes the horizontal path to scale (roughly 11,574 events/sec average for one billion events/day). The two are reported separately.
- "The LLM understands any log automatically." The agent only ever produces a Draft; deterministic tests and mandatory human review decide whether it becomes Active.

6. Scope

6.1 In Scope — MVP / Prototype

Priority	Source Class	Formats for MVP	Implementation
Must	Firewall	Syslog / proprietary text, JSON	Regex/key-value and JSONPath
Must	IDS/IPS	JSON, Syslog, CEF	JSONPath and CEF parser
High	VPN gateway	Syslog, JSON	Regex and JSONPath
High	WAF / proxy	JSON, text, CSV	JSONPath / regex / CSV
Medium	Router / switch	Syslog	Regex / key-value
Optional	LEEF / XML	LEEF, XML	Dedicated parser / XPath

Recommended project claim: "ULPF is extensible to new log sources through declarative, versioned parser rules and schema adapters. The implementation demonstrates the framework on perimeter-device telemetry in Syslog/text, JSON and CEF/LEEF-compatible formats."

Also in scope: the Immutable Raw Evidence Layer (MinIO/S3-compatible), the PostgreSQL Rule Registry with full lifecycle governance, the local Rule Authoring Agent (Qwen2.5-Coder-7B via llama.cpp, onboarding-only), sync/batch/streaming ingestion endpoints, the vendor portal (Studio, Rule Registry, Jobs & Sessions, API & Keys, Log Review), and OCSF/ECS export.

6.2 Out of Scope

- Detection rules, correlation logic, alerting and case management — these belong to the downstream SIEM.

- Long-term log retention and archival lifecycle management beyond the configured retention window.
- Log collection agents on endpoints — ULPF receives; it does not install collectors.
- Threat intelligence enrichment, geo-IP or asset enrichment — downstream enrichment concerns.
- Multi-tenant SaaS billing; multi-tenancy isolation for API-key scoping is in scope, hosted billing is not.
- Encrypted or binary proprietary formats requiring vendor decryption libraries.
- Fine-tuning or retraining the local model during the hackathon build — distillation is a roadmap item only.

6.3 Future Scope (post-MVP)

Phase	Capability	Notes
V1.1	Supervised distillation of a smaller local model	Trains only on reviewer-approved, Active rule versions — especially cases where a reviewer edited the agent's draft
V1.2	Rule package export / import	Share validated rules between air-gapped deployments
V1.3	Additional parser types	XPath/XML and further proprietary text dialects
V1.4	Kubernetes orchestration	Beyond the Docker Compose MVP
V2.0	Multi-region / multi-tenant vendor portal hardening	Deeper isolation for the API & Keys and Jobs & Sessions surfaces

7. Feature Requirements

Features are grouped into epics aligned with the seven pipeline stages plus onboarding, delivery and operations. Priority uses MoSCoW: M = Must have for the prototype, S = Should have, C = Could have. Each feature carries a forward reference to the functional requirement that specifies it in the SRS.

7.1 Epic F1 — Ingestion Gateway

ID	Feature	Description	Pri.	SRS ref.
F1.1	Syslog listener	UDP and TCP Syslog listeners on configurable ports	M	FR-1.1
F1.2	REST/gRPC API	Authenticated single-event (POST /v1/convert) and batch (POST /v1/jobs) endpoints	M	FR-1.2
F1.3	Streaming session	Session-based ingestion (POST /v1/sessions/{id}/events) sharing one detect/lock lifecycle	M	FR-1.3
F1.4	Batch file upload	File upload with per-job detection and locking	M	FR-1.4
F1.5	Source/tenant binding	Every ingestion channel and API key is bound to one log source/environment	M	FR-1.5
F1.6	Backpressure & buffering	Queue and shed load gracefully rather than dropping events silently	M	FR-1.6

7.2 Epic F2 — Detection & Rule Registry

ID	Feature	Description	Pri.	SRS ref.
F2.1	Format fingerprinting	Key-set hash (structured) or structural/delimiter signature (unstructured) computed before any parsing is attempted	M	FR-2.1

ID	Feature	Description	Pri.	SRS ref.
F2.2	Active-rule search	Fingerprint matched only against Active rules in the Rule Registry	M	FR-2.2
F2.3	Registry-miss routing	No match routes to the local Rule Authoring Agent rather than failing the event	M	FR-2.3
F2.4	Trust-but-verify	Explicit rule_id calls and locked batch/session rules are still validated per event or spot-check, never assumed correct	M	FR-2.4

7.3 Epic F3 — Local Rule Authoring Agent

ID	Feature	Description	Pri.	SRS ref.
F3.1	Sample intake	Analyst uploads 3–5 representative samples and selects a target schema (OCSF or ECS)	M	FR-3.1
F3.2	Secret redaction	Obvious secrets are redacted before any sample is used in a prompt	M	FR-3.2
F3.3	Declarative rule drafting	The local LLM returns JSON-only rule content: parser type, pattern/extraction, mappings, required fields, masking hints, fingerprints	M	FR-3.3
F3.4	Structural validation	The platform validates JSON structure, rejects forbidden content, compiles the parser in a sandbox and runs all test samples	M	FR-3.4
F3.5	Bounded retries	On deterministic validation failure, structured errors are returned to the LLM for at most 2–3 retries	M	FR-3.5
F3.6	Mandatory human review	A valid candidate is saved as Draft/Pending Review; a reviewer must approve it before Activation	M	FR-3.6
F3.7	Provenance capture	Model/runtime provenance, candidate output, validation runs and reviewer notes are recorded against the final rule version	M	FR-3.7

7.4 Epic F4 — Deterministic Parsing & Normalization

ID	Feature	Description	Pri.	SRS ref.
F4.1	Multi-parser support	Regex, JSONPath, CEF, LEEF, Syslog, key-value and XML/XPath parser types	M	FR-4.1
F4.2	Required-field validation	Checks required fields, types and target-schema compatibility before normalization	M	FR-4.2
F4.3	OCSF/ECS normalization	Maps parsed fields into the ULPF canonical model, then OCSF (primary) and ECS (adapter)	M	FR-4.3
F4.4	Unmapped-field preservation	Vendor-specific fields with no safe mapping are retained in unmapped_fields rather than dropped	M	FR-4.4
F4.5	Masking	Declarative masking policy (e.g. HMAC-SHA256) applied to configured sensitive fields	M	FR-4.5

7.5 Epic F5 — Immutable Raw Evidence Layer

ID	Feature	Description	Pri.	SRS ref.
F5.1	Raw-first write	Exact raw bytes are stored in MinIO/S3-compatible storage before any parsing touches them	M	FR-5.1

ID	Feature	Description	Pri.	SRS ref.
F5.2	Integrity digest	SHA-256 computed over the original bytes at write time	M	FR-5.2
F5.3	Reception metadata	event_id, timestamp, source transport metadata and byte length recorded alongside the raw object	M	FR-5.3
F5.4	Write-once storage	No processing path may modify or overwrite a stored raw object	M	FR-5.4
F5.5	Recoverability verification	A harness proving byte-identical recovery for every ingested event in a corpus	M	FR-5.5

7.6 Epic F6 — Traceability

ID	Feature	Description	Pri.	SRS ref.
F6.1	Event ID	A unique ID allocated at ingestion links raw, normalized and processing-history records	M	FR-6.1
F6.2	Rule provenance	Every normalized record carries rule_id, rule_version, rule_hash and schema_version	M	FR-6.2
F6.3	Raw ↔ normalized lookup	Bidirectional resolution between a raw event and the normalized event(s) derived from it	M	FR-6.3

7.7 Epic F7 — Rule Registry & Lifecycle

ID	Feature	Description	Pri.	SRS ref.
F7.1	Versioned rules	Every rule version is immutable and identified as name@version, e.g. cisco-asa-v1@2	M	FR-7.1
F7.2	Lifecycle states	Draft / Pending Review / Active / Deprecated / Disabled / Archived / Rejected, with auto-detect eligibility only for Active	M	FR-7.2
F7.3	One Active version per logical rule	Enforced by a partial unique index; old versions remain callable explicitly while Deprecated	M	FR-7.3
F7.4	Golden test cases	Every rule version carries test samples that must pass before Active	M	FR-7.4
F7.5	Rule package export/import	Rules and mapping packages export as files for transfer between air-gapped sites	S	FR-7.5

7.8 Epic F8 — Human-in-the-Loop Review

ID	Feature	Description	Pri.	SRS ref.
F8.1	Studio onboarding wizard	Guided flow: submit samples → view fingerprint/candidate rule → review → approve	M	FR-8.1
F8.2	Diff review	Reviewer sees the LLM's declarative rule diff, validation run results and test outcomes before approving	M	FR-8.2
F8.3	Approval audit trail	Records who approved which rule version, when, and any edits made	M	FR-8.3
F8.4	Log Review console	Inspect unresolved, ambiguous and format-drift events for manual triage	S	FR-8.4

7.9 Epic F9 — Downstream Delivery

ID	Feature	Description	Pri.	SRS ref.
F9.1	SIEM/data-lake export	Kafka/API export of normalized OCSF/ECS events	M	FR-9.1
F9.2	Analytics store	ClickHouse for high-volume normalized event queries	M	FR-9.2
F9.3	Search layer	OpenSearch for optional full-text SOC search and dashboards	S	FR-9.3
F9.4	Vendor API & Keys	Self-serve, source-scoped API keys with rotation and revocation	M	FR-9.4

7.10 Epic F10 — Deployment & Operations

ID	Feature	Description	Pri.	SRS ref.
F10.1	Single-command deployment	Docker Compose brings up the complete stack; Kubernetes is a later option	M	FR-10.1
F10.2	Air-gapped installation	All services, model weights (GGUF) and dependencies deploy locally in containers; no runtime downloads	M	FR-10.2
F10.3	Jobs & Sessions dashboard	Live progress and lock/spot-check status per batch job and streaming session	S	FR-10.3
F10.4	Health & metrics endpoints	Liveness, readiness and metrics for operational monitoring	S	FR-10.4

8. User Stories & Acceptance Criteria

US-01 — Onboard an unknown source

As a rule / detection engineer, I want to submit 3–5 samples from an appliance nobody has configured before and receive a declarative candidate rule, so that I can start producing normalized events without hand-writing a parser.

- AC-1 Submitting samples of a fingerprint with no Active-rule match invokes the local Rule Authoring Agent within the interactive response budget.
- AC-2 The candidate rule includes parser type, extraction pattern, field mapping, required fields, masking hints and fingerprints.
- AC-3 No source-specific executable code is written or deployed at any point in the flow — the artefact is declarative rule JSON only.
- AC-4 On approval, the rule is Activated and subsequent events from that fingerprint are processed on the deterministic fast path.

US-02 — Trust but verify a rule

As a rule / detection engineer, I want invalid or low-confidence candidates held for my review rather than activated silently, so that a wrong LLM draft never becomes production logic.

- AC-1 A candidate that fails deterministic structural or test validation is retried against the LLM at most 2–3 times, then held as Draft for manual authoring.
- AC-2 The reviewer can see the candidate's parser definition, field mapping, and the exact validation-run results and test outcomes.
- AC-3 A reviewer edit is recorded as a new version and the original candidate output is retained for provenance.
- AC-4 Events matching a Pending-Review fingerprint are still ingested and preserved raw; they are not normalized until a rule is Active.

US-03 — Recover the original event

As a compliance officer, I want to retrieve the exact original record behind any normalized event, so that I can present defensible evidence in an audit.

- AC-1 Given an `event_id`, the API returns the raw event byte-identical to what was received.
- AC-2 The response includes the SHA-256 digest computed at ingestion and re-verified at retrieval.
- AC-3 The response includes the exact `rule_id`, `rule_version`, `rule_hash` and `schema_version` applied.
- AC-4 Recoverability is demonstrated across the full test corpus, not sampled.

US-04 — Explain a normalized value

As an analyst, I want to know exactly which rule version produced a normalized event, so that I can judge whether to trust it during an investigation.

- AC-1 For any normalized event, the API returns the rule ID, version and hash used.
- AC-2 The response links to the raw object reference and its verified hash.
- AC-3 The response states whether the event was processed via explicit `rule_id`, auto-detect, or a locked batch/session rule.

US-05 — Nothing is lost

As a security engineer, I want vendor-specific fields that do not fit the common schema to be preserved, so that normalization does not destroy evidence.

- AC-1 Every field present in a parsed event either maps to a core OCSF/ECS field or appears under `unmapped_fields`.
- AC-2 Unmapped fields are queryable through the same API as core fields.
- AC-3 A schema-validation failure never silently emits an incomplete event — it returns an explicit unresolved/rejected result while retaining raw data.

US-06 — Run with no Internet

As an air-gapped network administrator, I want to install and operate ULPF, including the Rule Authoring Agent, on an isolated network, so that log data and log samples never leave my perimeter.

- AC-1 The offline bundle installs and starts with no outbound network access available.
- AC-2 The Rule Authoring Agent runs entirely from locally bundled GGUF model weights; no external inference call occurs at any point.
- AC-3 A network egress monitor observes zero outbound connections during a full onboarding-and-processing run.

US-07 — Survive a format change

As a platform engineer, I want a source that drifts from its locked rule to be caught rather than silently mis-parsed, so that firmware upgrades are surfaced as operational events, not silent data-quality incidents.

- AC-1 A background spot-check on a locked batch job or streaming session flags a mismatched event without failing the whole job.
- AC-2 After repeated mismatches, the lock is invalidated and the source returns to full per-event detection.
- AC-3 A drifted source that needs a new rule version follows the same LLM-draft → validate → review → activate path as first onboarding.

9. Competitive Landscape & Differentiation

This section exists because the single most dangerous question the team will face is "Splunk, Elastic and QRadar already do this — what is new?" The honest answer begins by conceding that those platforms are genuinely capable, and then states the specific wedge ULPF occupies.

9.1 Existing Platforms

Platform	How it solves normalization	Where the source-specific effort remains
Splunk Enterprise Security	Common Information Model (CIM) with technology add-ons per data source	Onboarding depends on an add-on per source; unusual sources need custom extraction
Elastic / Elastic Security	Ingest pipelines and processors normalizing into ECS	Arbitrary or unseen sources still require appropriate pipeline/processor configuration
IBM QRadar	Device Support Modules (DSMs) parse and normalize known sources	Unexpected formats may need a custom DSM built by hand in the DSM Editor

9.2 Capability Comparison

Capability	Established SIEM platforms	ULPF
Unknown / undocumented source	Custom parser, DSM or pipeline configuration is typically required, hand-authored	Fingerprint-miss triggers a local LLM to draft a declarative rule candidate for review
Rule authoring safety	Vendor-supplied or hand-written logic, reviewed by convention	The agent can only emit schema-validated declarative JSON, never executable code; validation is structural
Raw-event handling	Raw events are collected and retained	An explicit, hashed, write-once Raw Evidence Layer is a first-class architectural component
Rule provenance	Content-pack/DSM version metadata	Every event carries the exact rule_id, rule_version and rule_hash used
Air-gapped AI assistance	Not generally offered, or cloud-dependent where offered	The authoring LLM itself is local, quantized, and never calls out
Vendor independence	Ecosystem-centric by design	Framework-level objective; output feeds any downstream consumer

9.3 The Defensible Claim

ULPF's novelty is not any single component. Open schemas exist (OCSF, ECS). Local code LLMs exist (Qwen2.5-Coder). The contribution is the orchestration that connects fingerprint-based rule reuse, LLM-assisted declarative rule drafting, deterministic sandbox validation, mandatory human review, versioned rule lifecycle, lossless raw preservation and rule-hash provenance into a single, air-gapped preprocessing workflow — with the model kept strictly out of the production hot path.

10. Release Plan & Milestones

Milestone	Deliverable	Exit criteria
M0 — Foundation	Rule JSON schema, ULPF canonical model, Docker Compose, PostgreSQL/MinIO	An event received on any transport is stored, digested and retrievable by event_id
M1 — Deterministic path	Regex/key-value Syslog and JSONPath parsers for two or more formats	Known-fingerprint events normalize end to end via an Active rule

Milestone	Deliverable	Exit criteria
M2 — Registry governance	Rules, versions, fingerprints, test cases, approvals, lifecycle, audit	One Active version per logical rule enforced; lifecycle transitions recorded
M3 — Local Rule Authoring Agent	llama.cpp server, Qwen2.5-Coder-7B GGUF, JSON candidate generation, validation/retry	A registry miss produces a Draft candidate that passes structural validation or exhausts retries cleanly
M4 — Human review	Studio review UI, approval workflow, audit trail	A reviewer can approve, edit or reject a candidate; approval activates the rule
M5 — Batch/stream optimisation	Jobs/sessions, sampling, lock, spot-check and drift handling	A locked job fast-paths remaining events and flags a spot-check mismatch without failing the job
M6 — Analytics and delivery	ClickHouse/OpenSearch integration, SIEM export, vendor portal	Normalized events are queryable and exportable with full traceability
M7 — Hardening and evidence	Tests, benchmark, air-gap test, README, demo video, five-slide deck	Submission package complete per Section 19 of the Full Understanding Document

10.1 Demonstration Scenario

The prototype demonstration is built around the strongest moment: onboarding a source the system has never seen, entirely through the local agent. The scripted sequence is — present three heterogeneous logs (Syslog, JSON, CEF); upload an unknown Syslog sample in Studio and show no rule match; show the local agent's declarative rule draft; show deterministic validation and reviewer approval; send a single log and show the raw object/hash then the normalized event; open a batch/session and show the rule locked after sampling; inject a changed-format event and show the unresolved/format-drift flag; show a unified search/table across multiple vendors.

11. Assumptions, Dependencies & Constraints

11.1 Assumptions

- Log sources emit text-based or textually-representable events; encrypted or binary proprietary payloads requiring vendor libraries are out of scope.
- 3–5 representative samples are sufficient for the local agent to draft a candidate rule for a given format.
- Reviewers with domain knowledge are available to adjudicate every Draft/Pending-Review candidate; ULPF reduces their workload but does not eliminate the role.
- The deployment host provides sufficient CPU/RAM (and optionally a modest GPU) to run a quantized 7B GGUF model at interactive latency.

11.2 Dependencies

Dependency	Type	Consequence if unavailable
Qwen2.5-Coder-7B (GGUF) + llama.cpp	Local model + runtime	Rule authoring falls back to fully manual rule creation in Studio
PostgreSQL	Relational store	No durable Rule Registry, lifecycle governance or audit trail
MinIO / S3-compatible storage	Object store	No immutable raw evidence layer; the lossless guarantee cannot be made
OCSF / ECS specifications	External schema standards	Output schema would have to be proprietary, weakening downstream interoperability

Dependency	Type	Consequence if unavailable
Container runtime	Platform	Deployment story and air-gap packaging would require rework

11.3 Constraints

- No mandatory outbound network connectivity anywhere on the processing or authoring path.
- The prototype is built and evaluated within the hackathon timeline by a six-member student team; scope is prioritised accordingly.
- The Rule Authoring Agent's output is restricted to a declarative, schema-validated DSL — no scripts, shell commands, SQL, dynamic imports, URLs, credentials or network actions may ever appear in a rule.
- Benchmarks must be reproducible on commodity hardware that the team can specify and the jury could verify.

12. Risk Register

The product-level risk register below covers business and adoption risk. The detailed LLM failure-mode register (18 risks, spanning hallucinated mappings through resource exhaustion) is maintained in the SRS and TRD as an engineering artefact, since it drives specific validation and testing requirements.

ID	Risk	Impact	Likelihood	Mitigation
R1	An LLM-drafted rule is subtly wrong and is approved anyway	High	Medium	Golden-sample regression tests per rule version; reviewer sign-off required before Active; raw event always preserved
R2	Vendor-specific fields do not fit the common schema	Medium	High	unmapped_fields namespace plus retention of the original event — nothing is discarded
R3	Log formats change over time and drift from a locked rule	High	High	Adaptive-rate spot-checking during batch/stream locking; drift surfaces as an operational event, not silent re-interpretation
R4	Event volume exceeds single-node processing capacity	Medium	Medium	Fingerprint-narrowed candidates, batch/session locking, Kafka/Redpanda buffering, horizontally scalable containers
R5	Jury perceives the project as a generic log collector rather than a governed rule-authoring platform	High	Medium	Explicit positioning around declarative-rule safety, lifecycle governance and rule-hash traceability, not "AI parses anything"
R6	Unvalidated performance or accuracy claims are challenged	Medium	High	Targets are labelled as targets; measured results reported with stated hardware and workload
R7	On-box LLM latency or memory footprint is unacceptable on demo hardware	Medium	Medium	Quantized GGUF build sized to available RAM/VRAM; tested on the actual demo machine ahead of time

13. Glossary

Term	Definition
Parsing Rule	A versioned, declarative bundle: extraction pattern + field mapping + target schema + masking policy
Rule ID	A stable identifier combining name and version, e.g. cisco-asa-v1@2
Rule Authoring Agent	A local, quantized LLM (Qwen2.5-Coder-7B via GGUF/llama.cpp) invoked only on a registry miss during onboarding
Rule Registry	A versioned store of all rules in PostgreSQL — rules, versions, test cases, approvals and audit history
Ingestion Job	A batch upload of many events assumed to share one format
Ingestion Session	A streaming connection (session_id on every event) assumed to share one format
Format Fingerprint	A key-set hash (structured logs) or structural/delimiter signature (unstructured logs) used to match a sample before any parsing is attempted
Event ID	A unique ID assigned at ingestion, connecting the raw object, normalized outputs and processing history
Raw Hash	SHA-256 of the original received bytes, computed before decoding or transformation
Rule Hash	SHA-256 of the canonical rule content; proves the exact configuration used to parse an event
Unmapped Fields	Vendor-specific data retained in the normalized output when no safe common-schema mapping exists
Lifecycle State	Draft / Pending Review / Active / Deprecated / Disabled / Archived / Rejected
OCSF	Open Cybersecurity Schema Framework — the primary target taxonomy
ECS	Elastic Common Schema — an adapter/rendering profile over the same event
Air-gapped deployment	Operation inside a network with no route to the public Internet and no mandatory cloud or external API dependency, including for the authoring LLM